

How Text Becomes Numbers in NLP

Tokenization ▪ IDs ▪ Embeddings ▪ Context Windows ▪ Real LLM
Data Comparison

Your Name

2025-11-12

Why convert text to numbers?

- Neural networks operate only on numbers
- Text is symbolic
- We must encode it numerically

Pipeline:

Text $\rightarrow$ Tokens $\rightarrow$ IDs $\rightarrow$ Embeddings $\rightarrow$ Tensors

Predictive Modeling Analogy

Classical ML:

$$(X_1, X_2, \dots, X_k) \rightarrow y$$

Text modeling (next token prediction):

$$(t_{i-2}, t_{i-1}) \rightarrow t_i$$

Same idea: supervised learning.

Tokenization

Tokenization splits text into:

- Word tokens
- Subword tokens (BPE)
- Characters

Models do **not** understand words — they understand **token IDs**.

Example Sentence

I love large language models.

Tokens:

I
love
large
language
models
.

Vocabulary → Token IDs

Token	ID
I	1
love	2
large	3
language	4
models	5
.	6

Encoded sentence:

[1, 2, 3, 4, 5, 6]

Context Window

Assume:

- context length = 2
- predict the next token

Training examples:

$[I, \textit{love}] \rightarrow \textit{large}$

$[\textit{love}, \textit{large}] \rightarrow \textit{language}$

$[\textit{large}, \textit{language}] \rightarrow \textit{models}$

$[\textit{language}, \textit{models}] \rightarrow .$

Embedding Layer

Token IDs $\rightarrow$ embedding vectors

Embedding dimension = **3**

Token	Embedding
I	(0.1, 0.2, 0.3)
love	(-0.5, 0.1, 0.0)
large	(0.7, -0.2, 0.4)
language	(0.0, 0.3, 0.9)
models	(0.4, 0.4, 0.4)
.	(-0.2, 0.0, 0.1)

Converting Tokens to Embeddings

Example input:

[I, love] = [1, 2]

Embedding lookup:

1 $\rightarrow$ (0.1, 0.2, 0.3)

2 $\rightarrow$ (-0.5, 0.1, 0.0)

Input tensor (2×3):

$$\begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix}$$

Full Training Examples

Context length = 2, embed dim = 3:

$$[1, 2] \rightarrow \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix}$$

$$[2, 3] \rightarrow \begin{pmatrix} -0.5 & 0.1 & 0.0 \\ 0.7 & -0.2 & 0.4 \end{pmatrix}$$

$$[3, 4] \rightarrow \begin{pmatrix} 0.7 & -0.2 & 0.4 \\ 0.0 & 0.3 & 0.9 \end{pmatrix}$$

$$[4, 5] \rightarrow \begin{pmatrix} 0.0 & 0.3 & 0.9 \\ 0.4 & 0.4 & 0.4 \end{pmatrix}$$

Targets:

Next Token Prediction

The model learns:

$$f : \mathbb{R}^{2 \times 3} \rightarrow \text{Probability over tokens}$$

Example output:

- $P(\text{large}) = 0.72$
- $P(\text{language}) = 0.10$
- $P(\text{models}) = 0.08$

Prediction = argmax .

Regression Analogy

Text Concept	Regression Equivalent
Previous tokens	Input X
Next token	Output y
Token IDs	Encoded categorical vars
Embeddings	Learned numeric features
Context window	# predictors
Sequence model	Regression function

Comparing Tiny Dataset vs Real LLM Data

Our toy example:

- 6 tokens
- vocab size = 6
- context length = 2
- embed dim = 3
- 4 training samples

LLMs: trillions of tokens.

- 2–10 trillion tokens
- Books, web, code, math
- Embedding dim $\approx 10,000$
- Context $\approx 128\text{k}–1\text{M}$

- Trillions of tokens
- Heavy multimodal
- Embeddings $\approx 8\text{--}16\text{k}$
- Context $\approx 1\text{M}$

- Trillions of tokens
- Real-time X/Twitter
- Code, web, books
- Context $\approx$ 128k–1M

Summary: Tiny vs Massive

Feature	Example	ChatGPT	Gemini	Grok
Tokens	6	trillions	trillions	trillions
Vocab	6	~100k	~100k	~100k
Embedding dim	3	~10k	~10k	~8k
Context	2	1e5–1e6	1e6	1e5–1e6
Data size	1 sentence	internet-scale	internet- scale	internet- scale

Key Takeaways

- Same pipeline for small and huge models
- Only scale changes
- LLMs = same math, more compute

Questions?

Thank you!